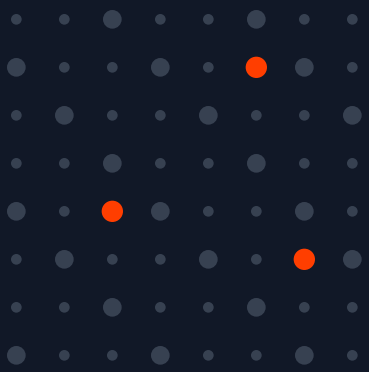
Svelte 5 Tutorial

A practical guide using real-world code

PROJECT: Sparrow Webhooks



Built with real examples from the Sparrow webhook platform codebase
github.com/sarathsp06/sparrow

Svelte 5 / TS

Table of Contents

Prerequisites: Basic JavaScript (ES6 level: var/let/const, arrow functions, promises, .map/.forEach). TypeScript syntax is explained inline as encountered.

Approach: Every example comes from the Sparrow webhook platform codebase with exact file paths and line numbers. Modern JS/TS syntax is explained as it appears.

Source Code: https://github.com/sarathsp06/sparrow

LESSON 1

Reactive State with $state()

The Problem

In a web app, data changes constantly: a user clicks a button, an API returns results, a timer fires. The UI needs to update to reflect these changes. In vanilla JavaScript, you'd manually find DOM elements and update their text content. Svelte automates this: you declare a variable as reactive, change it, and the UI updates itself.

The Svelte 5 Solution: $state()

The $state() rune tells the Svelte compiler to track a variable. When that variable's value changes (via plain assignment), every part of the UI that reads it re-renders automatically.

Source: health/+page.svelte, lines 15–21

```
let loading = $state(true );
let error = $state('');
let healthSummary = $state<HealthSummary | undefined>();
let unhealthyWebhooks = $state<RegisteredWebhook[]>([]);
let degradedWebhooks = $state<RegisteredWebhook[]>([]);
let webhookMetrics = $state(new Map<string, HealthMetrics>());
let namespaceStats = $state<NamespaceStats[]>([]);
```

Line by Line

- **let loading = $state(true)** — $state(true) creates a reactive variable initialized to true. While data is loading, the UI shows a skeleton placeholder. When loading completes, loading = false triggers the UI to swap to real content.
- **let error = $state('')** — An empty string means no error. If a fetch fails, error = 'Something went wrong' makes an error banner appear.
- **$state<HealthSummary | undefined>()** — The angle brackets are a **generic type parameter** (TypeScript). HealthSummary | undefined is a **union type**: the variable is either a HealthSummary object or undefined. Empty parentheses () mean no initial value = undefined.
- **$state<RegisteredWebhook[]>([])** — RegisteredWebhook[] means "an array of RegisteredWebhook objects". The [] inside parens is the initial value: empty array.

- **$state(new Map<string, HealthMetrics>())** — A Map is like a dictionary. Map<string, HealthMetrics> = keys are strings, values are HealthMetrics objects. Initialized empty.

How Reactivity Triggers

To update a $state variable, use plain assignment:

```
// This triggers a UI update:
loading = false ;
// Replacing the whole array:
unhealthyWebhooks = result.webhooks;
// For Maps, reassign the whole Map:
webhookMetrics = newMap;
```

The key insight: $state() is a **compiler directive**, not a regular function. At runtime, the variable holds the plain value (a boolean, a string, an array), not a wrapper object. Svelte's compiler rewrites your code during build to inject change tracking behind the scenes.

Gotcha 1: Mutating arrays

If you push to an array with .push(), Svelte 5 **does** detect the mutation (unlike Svelte 4 which required reassignment). However, reassignment is still the clearest pattern: items = [...items, newItem].

Gotcha 2: $state() is not a regular function

You cannot do const x = condition ? $state(1) : $state(2). The compiler must see $state() as a direct initializer in a let declaration at the top level of a component's script block.

LESSON 2

Computed Values with $derived()

The Problem

You have some state, and you need a value that's calculated from that state. For example: you have an array of webhooks and need a filtered list showing only the active ones. You want to declare the computation once and have it automatically stay in sync.

Simple Form: $derived(expression)

Source: CopyableId.svelte, lines 11-13

```
const display = $derived(
truncate > 0 && id.length > truncate
? id.substring(0, truncate) + '...'
: id
);
```

This creates a display value that automatically recomputes whenever truncate or id changes. The ternary operator (? :) checks: if truncation is enabled and the ID is longer than the limit, show a shortened version; otherwise show the full ID.

Complex Form: $derived.by(() => { ... })

Source: webhooks/+page.svelte, lines 45-57

```
let filteredWebhooks = $derived.by(() => {
let result = webhooks;
if (healthFilter) {
result = result.filter(
(wh) => wh.health === healthFilter
);
}
if (urlSearch.trim()) {
const q = urlSearch.toLowerCase();
result = result.filter((wh) =>
wh.url.toLowerCase().includes(q) ||
wh.description?.toLowerCase().includes(q)
);
}
```

```
return result;
});
```

Line by Line

- **`$derived.by(() => { ... })`** — When the computation needs multiple statements, use $derived.by() with a function body. The () => { ... } is an arrow function with a body that must explicitly return.
- **`.filter((wh) => wh.health === healthFilter)`** — .filter() creates a new array keeping only elements where the callback returns true.
- **`.includes(q)`** — A string method returning true if the string contains the substring. Case-insensitive search done by calling .toLowerCase() on both sides.
- **`wh.description?.toLowerCase()`** — The ?. is **optional chaining**. description might be undefined. Without ?., calling .toLowerCase() on undefined would crash. With it, it safely returns undefined instead.

$derived vs $state: When to Use Which

Use $state for data you set directly (API responses, user input). Use $derived for data computed from other state. If you find yourself writing code that sets a variable every time another changes, use $derived.

> **Gotcha 1: Only reactive sources are tracked**
>
> $derived only tracks variables from $state, $derived, or $props. A plain let x = 5 is not reactive.

> **Gotcha 2: Don't mutate inside $derived**
>
> A $derived computation should be pure. Never modify other $state variables inside. That's what $effect is for.

LESSON 3

Components & Props with $props()

The Problem

As your UI grows, you need to break it into reusable pieces. A health badge, a copyable ID display, a confirmation dialog. Each piece needs to accept data from its parent. In Svelte 5, components receive data through $props().

Defining Props with TypeScript

Source: CopyableId.svelte, lines 2-8

```
interface Props {
id: string;
href?: string;
truncate?: number;
}
let { id, href, truncate = 8 }: Props = $props();
```

Line by Line

- **interface Props { ... }** — A TypeScript **interface** defines the shape of an object. "Any object matching this type must have these properties with these types." The name Props is convention, not special to Svelte.
- **id: string** — Required prop. The component must receive a string id. Missing it = TypeScript compile error.
- **href?: string** — The ? makes this optional. Parent can pass it or omit it. When omitted, the value is undefined.
- **let { id, href, truncate = 8 }: Props = $props()** — Destructures props. truncate = 8 is a default value. The : Props tells TypeScript to type-check against the interface.

Callback Props (Functions as Props)

Source: ConfirmDialog.svelte, lines 1-8

```
interface Props {
show: boolean;
title: string;
message: string;
confirmLabel?: string;
cancelLabel?: string;
```

```
onconfirm: () => void;
oncancel: () => void;
}
```

- **`onconfirm: () => void`** — This prop expects a function that takes no arguments and returns nothing (void). The parent passes a callback. When the user clicks "Confirm", the dialog calls `onconfirm()`.

This is the standard pattern for child-to-parent communication in Svelte 5: the parent passes a function down, the child calls it when something happens.

Gotcha 1: Props are read-only

You cannot reassign a prop inside the child. `id = 'new'` would be a compile error. Props flow one direction: parent to child.

Gotcha 2: Default values only apply when undefined

If parent passes `truncate={0}`, default 8 does NOT apply. 0 is a valid value. Defaults only kick in when omitted or explicitly undefined.

LESSON 4

Conditional Rendering with {#if}

The Problem

Most UI isn't static. You need to show a loading spinner while data loads, an error message when something fails, and actual content when data arrives. Svelte's {#if} block controls what HTML is in the DOM.

Three-State Pattern: Loading / Error / Content

Source: health/+page.svelte, lines 97-146

```
{#if loading}
<!-- Skeleton placeholders -->
<div class="animate-pulse">...</div>
{:else if error}
<!-- Error message -->
<div class="text-red-600">{error}</div>
{:else}
<!-- Actual content -->
<div>{healthSummary.totalWebhooks}</div>
{/if}
```

How It Works

- **{#if loading}** — Opens a conditional block. If loading is truthy (not false, 0, '', null, undefined, or NaN), this branch renders.
- **{:else if error}** — If loading was falsy, check error. An empty string '' is falsy (no error), a non-empty string is truthy (there is an error).
- **{:else}** — The fallback. If neither loading nor error is truthy, show the real content.
- **{/if}** — Closes the block. Every {#if} must have a matching {/if}.

DOM Destruction vs CSS Hiding

{#if} **removes elements from the DOM** when the condition is false. It doesn't hide them with CSS. When true again, Svelte creates fresh DOM elements. Any internal state (scroll position, input values) is lost.

Inline Ternaries in Attributes

```
<span class="{wh.active ? 'bg-green-500' : 'bg-gray-300'}">
{wh.active ? 'Active' : 'Paused'}
</span>
```

Use ternaries for attribute variations (colors, text). Use {#if} when entire HTML sections need to appear/disappear.

Gotcha 1: Equality checks

JavaScript's == does type coercion ("5" == 5 is true). Always use === for strict equality.

Gotcha 2: Empty arrays are truthy

An empty array [] is truthy. {#if myArray} is always true. Use {#if myArray.length > 0} instead.

LESSON 5

Rendering Lists with {#each}

The Problem

You have an array of data and need to render each item. Svelte's {#each} block iterates over an array and renders a template for each.

Basic Usage: Table Rows

Source: webhooks/+page.svelte, lines 484-512

```
{#each filteredWebhooks as wh}
<tr onclick={() => goto(`/webhooks/${wh.webhookId}`)}>
<td>{wh.url}</td>
<td>
<CopyableId id={wh.webhookId}
href="/webhooks/{wh.webhookId}" />
</td>
{#each wh.events.slice(0, 2) as event}
<span class="badge">{event}</span>
{/each}
{#if wh.events.length > 2}
<span>+{wh.events.length - 2}</span>
{/if}
</tr>
{/each}
```

Key Points

- **{#each filteredWebhooks as wh}** — filteredWebhooks is the array. wh is the loop variable holding one webhook object per iteration.
- **wh.events.slice(0, 2)** — Nested {#each} inside the outer loop. .slice(0, 2) returns the first 2 elements, creating a "show first 2 + overflow count" pattern.
- **{#if wh.events.length > 2}** — After the inner loop, show a '+N' badge if there are more events. Mixing {#each} and {#if} is common and natural.

Skeleton Placeholders with Array(n)

Source: health/+page.svelte, lines 104-109

```
{#each Array(4) as _}
```

```
<div class="bg-white animate-pulse">
<div class="h-8 bg-gray-200 rounded"></div>
<div class="h-4 bg-gray-100 rounded"></div>
</div>
{/each}
```

- **`Array(4)`** — Creates an array with 4 empty slots. We just need the loop to repeat 4 times. This is the standard idiom for "repeat N times" since Svelte has no for loop.
- **`as _`** — The underscore `_` means "I don't use this value." Each element is undefined.

Gotcha 1: Keyed vs Unkeyed Lists

Default {#each} is unkeyed (reuses DOM by position). For reorderable lists with inputs, add a key: {#each items as item (item.id)}.

Gotcha 2: Destructuring in the loop

You can destructure: {#each webhooks as { url, webhookId }}. Sparrow prefers wh.url for clarity with many-property objects.

LESSON 6

Inline Constants with {@const}

The Problem

Inside an {#each} loop or {#if} block, you often need to compute a value. Without {@const}, you'd duplicate the expression everywhere or pre-compute it in the script section (overkill for template-only values).

Map Lookup

Source: health/+page.svelte, line 206

```
{#snippet webhookCard(wh: RegisteredWebhook)}
{@const metrics = webhookMetrics.get(wh.webhookId)}
<a href="/webhooks/{wh.webhookId}">
{#if metrics}
<span>{(metrics.successRate * 100).toFixed(1)}%</span>
<span>{metrics.failedDeliveries}</span>
{/if}
</a>
{/snippet}
```

- **{@const metrics = webhookMetrics.get(wh.webhookId)}** — Creates a block-scoped constant. .get() is the Map method returning the value or undefined. Without {@const}, you'd repeat the .get() call 5+ times.

Computed Value for Rendering

Source: health/+page.svelte, line 228

```
{@const totalErrors = (metrics.clientErrors || 0)
+ (metrics.serverErrors || 0)
+ (metrics.timeoutErrors || 0)
+ (metrics.networkErrors || 0)
+ (metrics.unexpectedStatusErrors || 0)}
{#if totalErrors > 0}
<div style="width: {(metrics.clientErrors / totalErrors) * 100}%">
</div>
{/if}
```

- **(metrics.clientErrors || 0)** — The || 0 fallback prevents NaN from undefined + undefined.

Function Call Result

Source: deliveries/[deliveryId]/+page.svelte, line 136

```
{#if delivery.errorCategory !== 'success'}
{@const cat = getCategoryDisplay(delivery.errorCategory)}
<span class="{cat.bgColor} {cat.color}">
{cat.label}
</span>
{/if}
```

Calls the function once, stores the result, then uses cat.bgColor, cat.color, cat.label — four references to one function call.

Reactivity Behavior

{@const} is **not** reactive on its own. It re-evaluates when its surrounding block re-renders. If webhookMetrics ($state Map) changes, Svelte re-renders the snippet, and the {@const} line runs again with new Map contents.

Gotcha 1: Must be at the top of a block

{@const} must be first in its block (or after another {@const}). Cannot be placed between two div elements mid-block.

Gotcha 2: Truly const

Cannot reassign it. Mutable values belong in the script section as $state.

Gotcha 3: No async

{@const} is synchronous only. Cannot use await. All async data fetching happens in the script section, stored in $state. Templates only do synchronous reads.

LESSON 7

Event Handling

The Problem

Users click buttons, type in inputs, press Enter, select dropdowns. In Svelte 5, event handlers are plain HTML attributes — no special syntax. If you know DOM events, you know 80% of this.

Inline Handler

Source: webhooks/+page.svelte, line 487

```
<tr
onclick={() => goto(`/webhooks/${wh.webhookId}`)}
>
```

- **onclick** — Standard HTML attribute, lowercase. In Svelte 5, pass a function. Different from Svelte 4's on:click (colon syntax is deprecated).
- **() => goto(...)** — Arrow function calling goto(), SvelteKit's navigation function. No event object needed — we just care that the click happened.

Handler with Event Object

Source: webhooks/+page.svelte, line 548

```
<button
onclick={(e) => toggleActive(wh, e)}
>
```

- **(e) => toggleActive(wh, e)** — Receives the native DOM event object. The handler likely calls e.stopPropagation() to prevent bubbling to the parent tr's onclick.
- **Closure over wh** — wh comes from the {#each} loop. Each row's handler captures its own wh. Standard JavaScript closure behavior.

Named Function Reference

Source: CopyableId.svelte, lines 15-25 & 42

```
// In <script>:
async function copyId(e: Event) {
```

```
e.stopPropagation();
e.preventDefault();
try {
await navigator.clipboard.writeText(id);
copied = true ;
setTimeout(() => { copied = false ; }, 1500);
} catch {
// Fallback: noop
}
}
// In template:
<button onclick={copyId}>
```

- **onclick={copyId}** — Passes the function reference. No parentheses — copyId not copyId(). Svelte passes the native event as the first argument.
- **e.stopPropagation()** — Prevents the click from bubbling to parent elements.
- **e.preventDefault()** — Prevents default browser behavior (e.g., navigation if inside an <a> tag).
- **await navigator.clipboard.writeText(id)** — Browser clipboard API. Async because the browser may need permission. await pauses until the copy completes.
- **copied = true** — A $state variable. Setting it triggers a re-render, swapping the copy icon for a green checkmark.
- **setTimeout(() => { copied = false }, 1500)** — After 1.5 seconds, reset the icon. setTimeout runs the callback after the delay (milliseconds).

Keyboard Events

Source: deliveries/+page.svelte, line 267

```
<input
bind:value={namespaceFilter}
onkeydown={(e) => e.key === 'Enter' && applyFilters()}
/>
```

- **onkeydown** — Fires on every key press while the input is focused.
- **e.key === 'Enter' && applyFilters()** — The && short-circuit trick. A && B evaluates B only if A is truthy. If key is Enter, call applyFilters(). Otherwise, do nothing.

Form Submission

Source: events/[eventName]/update/+page.svelte, line 131

```
<form onsubmit={updateEvent}>
<input bind:value={name} disabled />
<textarea bind:value={description}></textarea>
<button type="submit">Update Event</button>
</form>
```

- **onsubmit={updateEvent}** — Fires on form submission. The handler calls e.preventDefault() to stop the browser's default form POST, handling it with JavaScript instead.

Change Events on Selects

Source: deliveries/+page.svelte, line 276

```
<select
bind:value={statusFilter}
onchange={applyFilters}
>
<option value="">All</option>
</select>
```

- **onchange={applyFilters}** — Fires when the user picks a different option. bind:value updates statusFilter, then onchange triggers applyFilters() to re-fetch data.

  Gotcha 1: handler vs handler()

  onclick={handler} passes the function (runs on click). onclick={handler()} CALLS it immediately during render. Almost always a bug.

Gotcha 2: Svelte 5 vs Svelte 4 syntax

Old tutorials: on:click={handler}. Svelte 5: onclick={handler}. The colon syntax still works but is deprecated. Our codebase uses the new style.

Quick Reference Card

Concept	Syntax	Use When
Reactive state	`$state(initialValue)`	Data you set directly (API results, user input)
Computed value	`$derived(expression)` `$derived.by(() => { })`	Value calculated from other state
Component props	`let { a, b } = $props()`	Receiving data from parent component

Conditional	`{#if cond}`
	`{:else if}`
	`{:else}`
	`{/if}`
		Showing/hiding entire DOM sections

List rendering	`{#each array as item}` `{#each arr as x (key)}`	Rendering arrays. Add (key) for reorder
Inline constant	`{@const x = expr}`	Avoiding repeated expressions in templates
Event handler	`onclick={handler}` `onclick={(e) => { }}`	Responding to user interactions

Key TypeScript Syntax Explained in This Tutorial

Syntax	Meaning	Example
`x: string`	Type annotation	`let name: string = 'hi'`
`x?: string`	Optional property	`interface { href?: string }`
`A | B`	Union type (A or B)	`string | undefined`
`T[]`	Array of type T	`RegisteredWebhook[]`
`Map<K, V>`	Map with key/value types	`Map<string, Metrics>`
`() => void`	Fn, no args, no return	`onconfirm: () => void`
`x?.y`	Optional chaining	`desc?.toLowerCase()`
`<T>()`	Generic type parameter	`$state<Webhook[]>([])`
`interface`	Object shape definition	`interface Props { ... }`

All code examples from the Sparrow webhook platform.

https://github.com/sarathsp06/sparrow